Insurance AI Assistant

Version 2 — Debug Session Changelog & Root-Cause Log

This document logs every bug found after Version 1's initial build, the root cause of each, and the exact code change applied. Written as a direct continuation of **version1.pdf**, for the same purpose: handing the project to anyone (including a fresh AI session) to pick up cold.

---

1. What This Session Covered

Version 1 was reported as "fully working end-to-end." In practice, testing with a second uploaded document (a short FAQ alongside the 102-page compendium) surfaced six separate bugs, layered on top of each other — several masked one another, which is why they took multiple rounds of debugging to isolate. All six are fixed in this version.

#	Symptom	Root Cause	Fixed In
1	Bot answer cut off mid-word ("A: Term insur")	Context sent to LLM was hard-capped at 350 chars — sliced through the answer text itself	prompt_builder.py
2	Vector store always empty after server restart	vector_db_path was a relative path — resolved differently depending on terminal's cwd	config.py
3	New doc broke retrieval for unrelated questions	TOC/index pages misread as chapter headers, tagging junk "plan" names onto chunks	pdf_loader.py
4	Plan-filtered search returned zero results	No fallback when a guessed plan filter was wrong — search starved instead of retrying	chroma_client.py
5	Follow-up question inherited wrong plan scope	Plan detection ran on previous+current question combined, not current alone	chat_service.py
6	"What is X?" wrongly fell back (no bug)	Source documents genuinely lacked a plain-English definition — correct behavior	content, not code

2. Bugs — Detail, Root Cause, Fix

2.1 Context truncation cut answers mid-word

Symptom: Bot replied with fragments like **"A: Term insur"** instead of a full answer.

Root cause: _format_context() capped every retrieved chunk to a flat 350 characters, originally added to stay under Groq's free-tier token limit. On chunks where the relevant Q&A landed near the end, the cut sliced straight through mid-sentence, mid-word.

File: app/prompts/prompt_builder.py

Before:

```
text = c["text"][:350]  # cap per-chunk length to stay under token limits
```

After:

```
text = c["text"]
cap = 900
if len(text) > cap:
    truncated = text[:cap]
    last_period = truncated.rfind(". ")
    text = truncated[:last_period + 1] if last_period > cap * 0.5 else truncated
```

900-char cap (up from 350) with sentence-boundary cutoff instead of a raw slice. Even at top_k=10, worst case is ~9000 chars (~2,250 tokens) of context — well inside limits once combined with the max_completion_tokens fix in 2.6 freeing up output budget.

2.2 Vector store pointed at the wrong folder

Symptom: After a server restart, every query returned **zero chunks** and fell back, even for previously-working questions.

Root cause: vector_db_path defaulted to the relative path **"./vector_db"**, which ChromaDB resolves against the terminal's current working directory at launch time — not the project folder. Starting uvicorn from a different shell/cwd silently pointed it at a new, empty (or wrong-nested) folder.

File: app/config.py

Before:

```
vector_db_path: str = "./vector_db"
```

After:

```
import os
BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))

class Settings(BaseSettings):
    ...
    vector_db_path: str = os.path.join(BASE_DIR, "vector_db")
```

Path is now derived from the location of config.py itself, so it always resolves to the same folder regardless of which terminal/cwd launches the server.

2.3 Table-of-contents pages mistagged as plan headers

Symptom: Uploading a second (larger) document caused unrelated questions to return the fallback message, even when the answer clearly existed in a different, unrelated PDF.

Root cause: HEADER_PATTERN was built to catch chapter headers like "2. Family Floater Health Plan" — but it also matched every line of a table-of-contents/index page (e.g. "31 Auto Rickshaw Insurance"). Those got tagged as fake "plan" names. Once enough junk plan names existed, any later question sharing a word with one of them got wrongly scoped to that plan's chunks only.

File: app/services/pdf_loader.py

Before:

```
match = HEADER_PATTERN.search(raw)
if match:
    current_plan = match.group(1).strip()
```

After:

```
matches = HEADER_PATTERN.findall(raw)
# a real chapter-header page has exactly 1 match;
# a TOC/index page lists many -> skip tagging
if len(matches) == 1:
    current_plan = matches[0].strip()
```

2.4 Plan filter could starve retrieval to zero

Symptom: Even after fix 2.3, a wrong plan guess could still return zero chunks for a genuinely answerable question.

Root cause: similarity_search() applied the plan filter with no fallback: if the guessed plan name didn't actually cover the right content, ChromaDB's **where** clause simply excluded everything else, and retrieval failed silently.

File: app/vectorstore/chroma_client.py

Before:

```
results = collection.query(
    query_embeddings=[query_embedding],
    n_results=top_k,
    where=where_filter,
)
# (used directly, no fallback)
```

After:

```
results = collection.query(..., where=where_filter)
docs = results.get("documents", [[]])[0]

if not docs and where_filter:
    # plan filter guessed wrong -> retry unfiltered
    # instead of starving retrieval entirely
    results = collection.query(query_embeddings=[query_embedding], n_results=top_k)
    docs = results.get("documents", [[]])[0]
    ...
```

2.5 Previous question's plan filter leaked into the next question

Symptom: Asking about Plan A, then immediately asking an unrelated question, returned fallback for the second question even though its answer existed in a different part of the documents.

Root cause: search_query intentionally folds the previous question into the current one, so follow-ups like "what's the coverage for it" can resolve pronouns. But **detect_plan()** was run against that same combined string — so the previous question's plan name kept matching and locking every subsequent question to the wrong scope.

File: app/services/chat_service.py

Before:

```
search_query = f"{history[-1]['question']} {question}" if history else question
plan = detect_plan(search_query, plans)
```

After:

```
# plan lock must come from the CURRENT question only
plan = detect_plan(question, plans)

# history-folding kept only for the embedding/search text itself
```

```
search_query = f"{history[-1]['question']} {question}" if history else question
```

2.6 LLM output truncation (secondary contributor)

Symptom: Same symptom as bug 2.1 before the real cause (350-char context cap) was identified.

Root cause: openai/gpt-oss-20b is a reasoning model that can spend its token budget on hidden chain-of-thought before writing the final answer. No max_completion_tokens was set, so under some conditions the visible answer could be cut short independent of the context-cap issue.

File: app/services/llm_service.py

Before:

```
response = client.chat.completions.create(
    model=MODEL,
    messages=[{"role": "user", "content": prompt}],
)
```

After:

```
response = client.chat.completions.create(
    model=MODEL,
    messages=[{"role": "user", "content": prompt}],
    max_completion_tokens=1024,
    reasoning_effort="low",
)
```

Kept as a defensive fix even after confirming the 350-char cap was the primary cause — cheap insurance against the reasoning model burning its budget on unnecessary chain-of-thought.

3. Confirmed NOT a Bug: Missing Definitions

Early in this session, "What is term insurance?" fell back to the no-answer message even though it felt like it should work. Investigation (dumping every stored chunk directly) showed the uploaded FAQ document genuinely did not contain a plain-language definition of term insurance — only a compendium entry with policy-schedule numbers (sum insured, premium), not a conceptual explanation.

The fallback was **correct** per the project's own anti-hallucination rule: never answer from outside the provided context. Resolution was a content fix, not a code fix — the FAQ document was updated to include explicit Q&A entries for *term insurance*, *whole life insurance*, and *third-party insurance*, then re-indexed.

Operational lesson: re-uploading a changed PDF under the same filename does not cleanly replace old content. Chunk IDs are positional (filename_page_chunkid); ChromaDB's add() skips any ID that already exists rather than overwriting it. If the edit shifts chunk boundaries, stale text can persist under old IDs. Always delete a source's existing chunks before re-uploading an edited version of the same document:

```
from app.vectorstore.chroma_client import get_collection

c = get_collection()
existing = c.get(where={"source": "<filename>.pdf"}, include=[])
if existing["ids"]:
    c.delete(ids=existing["ids"])
```

4. Architecture — What Changed vs. Version 1

No structural/pipeline changes. Same 8-layer flow as Version 1 (Browser -> FastAPI -> Chat Service -> Embedding -> ChromaDB -> Prompt Builder -> Groq LLM -> Validation -> MySQL). All fixes are internal hardening of existing stages:

Stage	Version 1 Behavior	Version 2 Behavior
Prompt Builder	Hard 350-char cut per chunk	900-char cap, cuts on sentence boundary
Config	Relative vector_db_path	Absolute path anchored to project root
PDF Ingestion	Any numbered line -> plan tag	Only single-match header pages -> plan tag
Vector Search	Plan filter, no fallback	Plan filter with unfiltered retry on empty result
Chat Orchestration	Plan detected from question+history	Plan detected from current question only
LLM Call	No output token/effort control	max_completion_tokens + reasoning_effort set

5. Debugging Approach Used This Session

The lesson from Version 1 (semantic search alone doesn't scale to near-duplicate content) held again here, compounded by a second lesson: **bugs can stack and mask each other.** The fix sequence that worked:

• Never trust a symptom at face value — "fallback message" had three unrelated causes across this session (empty store, bad plan filter, and genuinely missing content).

• Isolate each pipeline stage independently with small debug scripts (direct similarity_search calls, raw chunk dumps, raw fitz extraction) rather than only testing through the full chat endpoint — narrows down which of the 8 stages actually failed.

• Verify file state on disk explicitly (Get-Item timestamps/length) before assuming an upload used the version of the file you think it did — copy/upload ordering mistakes were a repeat source of confusion.

• When re-testing after a fix, re-run the exact same debug script rather than only the chat UI — confirms the fix at the layer it targets, not just the end symptom.

6. Current Status — End of Session

✓ All 6 identified bugs fixed and individually verified via direct debug scripts.

✓ Cross-document retrieval confirmed working (FAQ doc + 102-page compendium coexist correctly).

✓ Follow-up/plan-scoping confirmed not to leak between unrelated consecutive questions.

✓ FAQ document content gap (term/whole life/third-party insurance) closed and re-indexed.

✓ Remaining recommended step: delete+reindex is now the standard procedure for any future document content edit, to avoid the stale-chunk-ID issue documented in Section 3.